

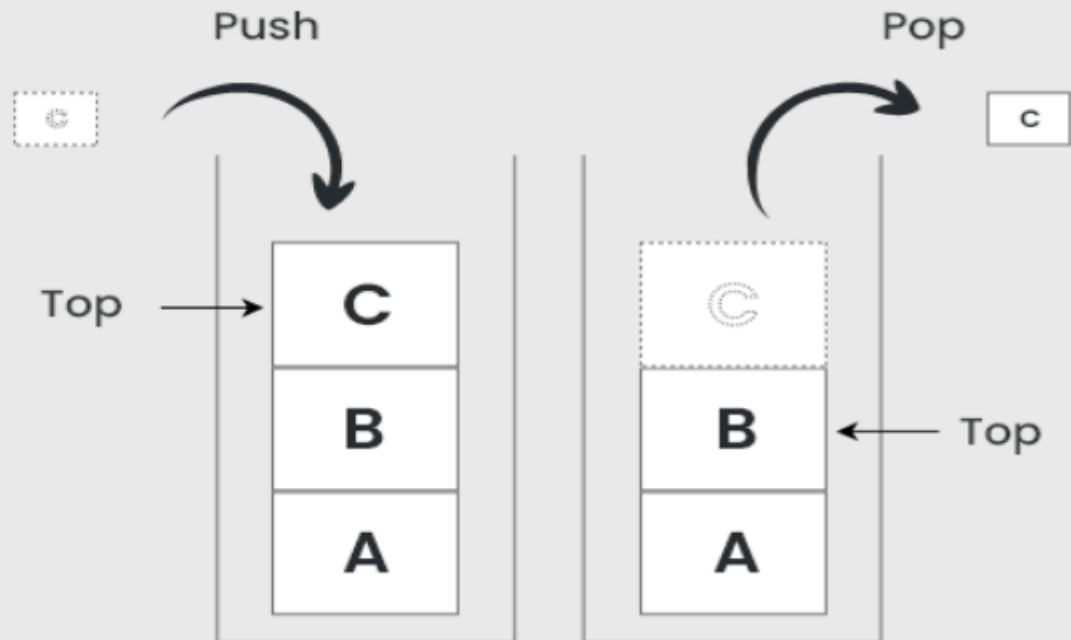
Unit 2

Stack

- A **Stack** is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO (Last In First Out)** or **FILO (First In Last Out)**. **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.

Stack

Data Structure

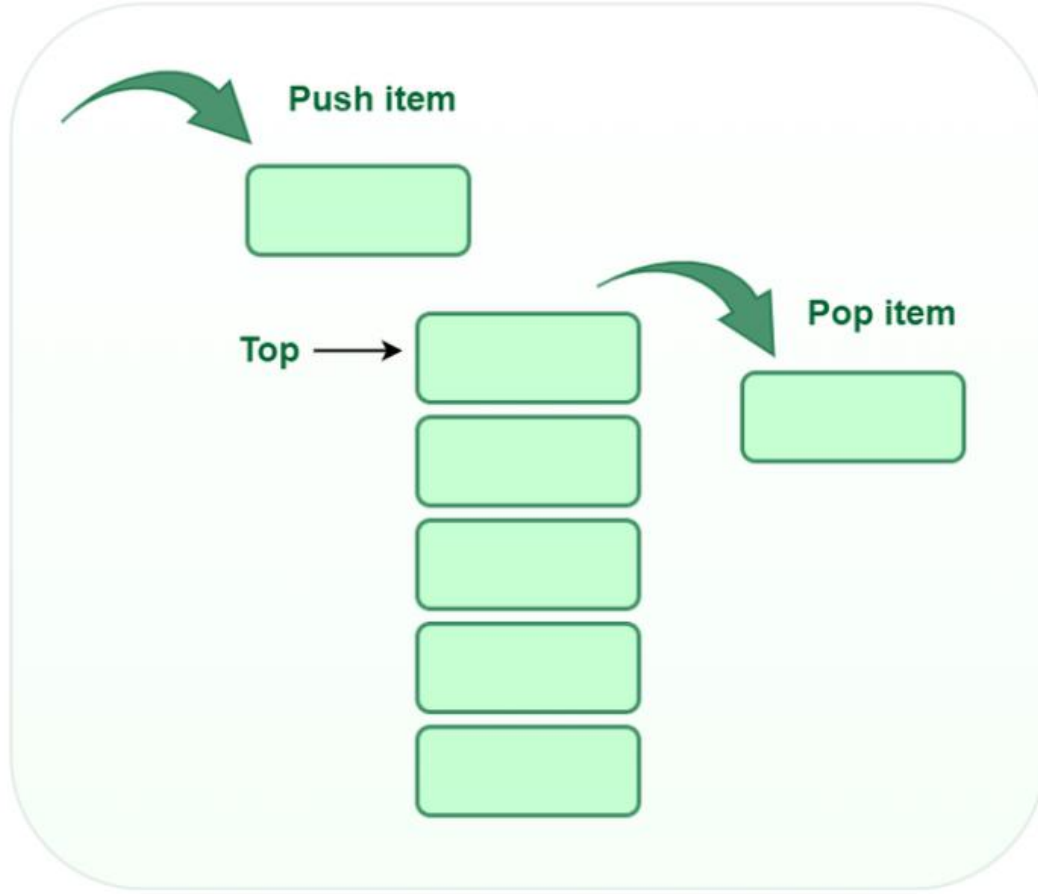


Operations on Stack

- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if the stack is empty else false.
- **size()** returns the size of the stack.

Applications of Stack Data Structures

- Recursion
- Expression Evaluation and Parsing
- Depth-First Search (DFS)
- Undo/Redo Operations
- Browser History
- Function Calls



Push Operation in Stack

- *Push operation adds an item to the stack*

Convert Infix expression to Postfix expression

- *Input:* $A + B * C + D$
Output: ABC^*+D+
- *Input:* $((A + B) - C * (D / E)) + F$
Output: $AB+CDE/*-F+$

Why?

- The compiler scans the expression either from left to right or from right to left.
Consider the expression: **$a + b * c + d$**
- The compiler first scans the expression to evaluate the expression $b * c$, then again scans the expression to add a to it.
- The result is then added to d after another scan.
- The repeated scanning makes it very inefficient. Infix expressions are easily readable and solvable by humans whereas the computer cannot differentiate the operators and parenthesis easily so, it is better to convert the expression to postfix(or prefix) form before evaluation.

Below are the steps

1. Scan the infix expression **from left to right**.

2. If the scanned character is an operand, put it in the postfix expression.

3. Otherwise, do the following

- If the precedence and associativity of the scanned operator are greater than the precedence and associativity of the operator in the stack [or the stack is empty or the stack contains a '('], then push it in the stack. ['^' operator is right associative and other operators like '+', '-', '*' and '/' are left-associative].
 - Check especially for a condition when the operator at the top of the stack and the scanned operator both are '^'. In this condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.
 - In all the other cases when the top of the operator stack is the same as the scanned operator, then pop the operator from the stack because of left associativity due to which the scanned operator has less precedence.
- Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator.
 - After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)

- 4.If the scanned character is a '(', push it to the stack.
- 5.If the scanned character is a ')', pop the stack and output it until a '(' is encountered, and discard both the parenthesis.
- 6.Repeat steps **2-5** until the infix expression is scanned.
- 7.Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.
- 8.Finally, print the postfix expression.

- $a+b*c+d$

- $a + b * (c^d - e)^{(f + g * h)} - i$

- $K + L - M * N + (O^P) * W / U / V * T + Q$

- $abcd^e-fgh^*+^*+i-$

Expression evaluation

- **Infix expression : $2 * (5 * (3 + 6)) / 5 - 2$**

- **Postfix expression : 2 5 3 6 + * * 5 / 2 –**
- **Prefix expression: - / * 2 * 5 + 3 6 5 2**
- **Reversed prefix expression: 2 5 6 3 + 5 * 2 * / -**

Queue Data Structure

- A queue is a linear data structure that follows the **First-In-First-Out (FIFO)** principle. It operates like a line where elements are added at one end (**rear**) and removed from the other end (**front**).

Queue Data Structure



Basic Operations of Queue Data Structure

- **Enqueue (Insert):** Adds an element to the rear of the queue.
- **Dequeue (Delete):** Removes and returns the element from the front of the queue.
- **Peek:** Returns the element at the front of the queue without removing it.
- **Empty:** Checks if the queue is empty.
- **Full:** Checks if the queue is full.

Applications of Queue

- **Task scheduling** in operating systems
- **Data transfer** in network communication
- **Simulation** of real-world systems (e.g., waiting lines)
- **Priority queues** for event processing queues for event processing

enqueue() and dequeue()

```
• #include <bits/stdc++.h>
• using namespace std;

• struct QNode {
•     int data;
•     QNode* next;
•     QNode(int d)
•     {
•         data = d;
•         next = NULL;
•     }
• };

• struct Queue {
•     QNode *front, *rear;
•     Queue() { front = rear = NULL; }

•     void enqueue(int x)
•     {
•         // Create a new LL node
•         QNode* temp = new QNode(x);

•         // If queue is empty, then
•         // new node is front and rear both
•         if (rear == NULL) {
•             front = rear = temp;
•             return;
•         }

•         // Add the new node at
•         // the end of queue and change rear
•         rear->next = temp;
•         rear = temp;
•     }

•     // Function to remove
```

enQueue()

```
• // a key from given queue q
• void deQueue()
• {
•     // If queue is empty, return NULL.
•     if (front == NULL)
•         return;

•     // Store previous front and
•     // move front one node ahead
•     QNode* temp = front;
•     front = front->next;

•     // If front becomes NULL, then
•     // change rear also as NULL
•     if (front == NULL)
•         rear = NULL;

•     delete (temp);
• }
• };
```

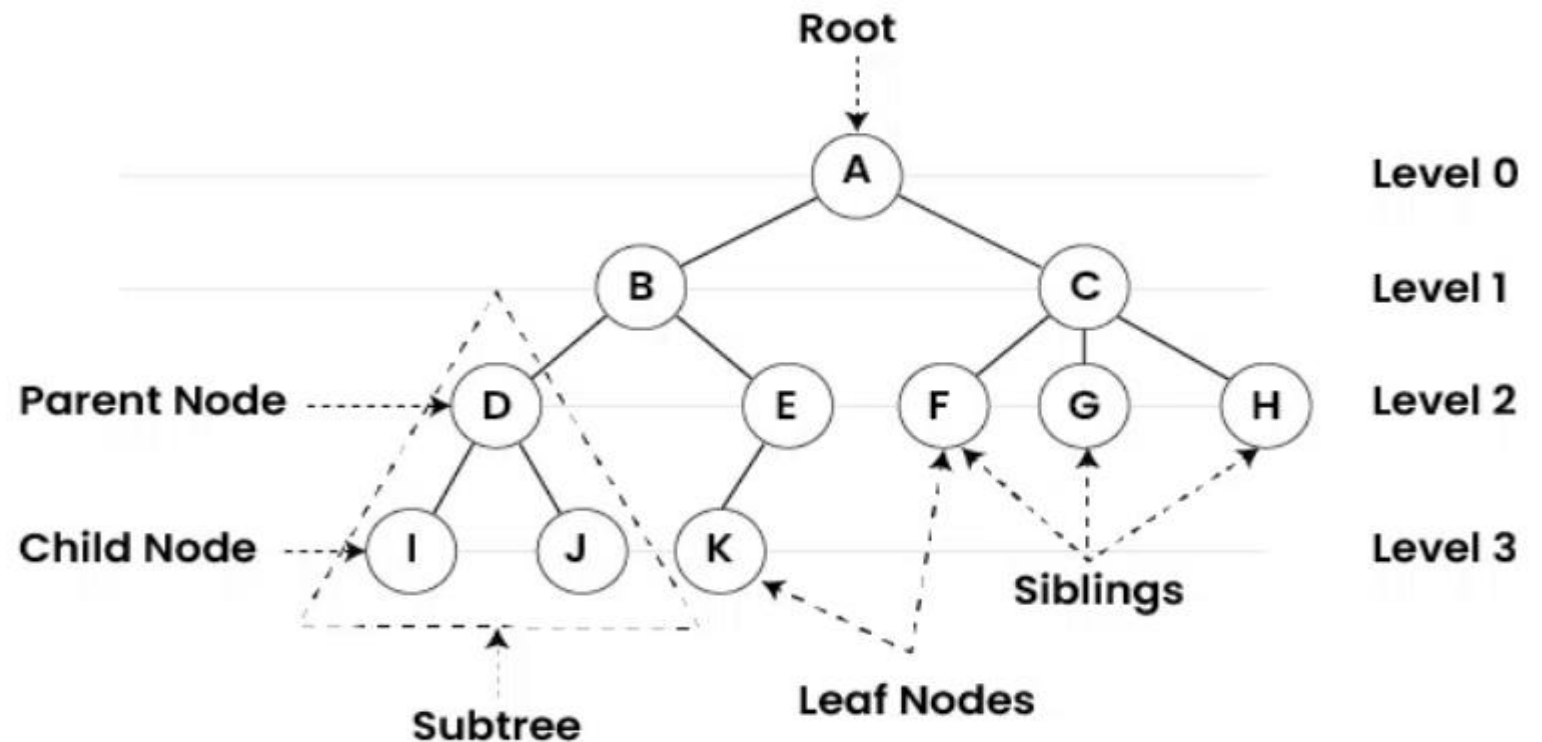
```
• // Driver code
• int main()
• {
•     Queue q;
•     q.enQueue(10);
•     q.enQueue(20);
•     q.deQueue();
•     q.deQueue();
•     q.enQueue(30);
•     q.enQueue(40);
•     q.enQueue(50);
•     q.deQueue();
•     cout << "Queue Front : " << ((q.front != NULL) ? (q.front)->data
• : -1)<< endl;
•     cout << "Queue Rear : " << ((q.rear != NULL) ? (q.rear)->data : -
• 1);
• }
```

Tree Data Structure

- **Tree Data Structure** is a non-linear data structure in which a collection of elements known as nodes are connected to each other via edges such that there exists exactly one path between any two nodes.

Tree

Data Structure



Terminologies In Tree Data Structure

- **Parent Node:** The node which is a predecessor of a node is called the parent node of that node. {B} is the parent node of {D, E}.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: {D, E} are the child nodes of {B}.
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. {K, L, M, N, O, P, G} are the leaf nodes of the tree.
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. {A,B} are the ancestor nodes of the node {E}
- **Descendant:** A node x is a descendant of another node y if and only if y is an ancestor of x.
- **Sibling:** Children of the same parent node are called siblings. {D,E} are called siblings.
- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level 0.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbor of a Node:** Parent or child nodes of that node are called neighbors of that node.
- **Subtree:** Any node of the tree along with its descendant.

Types of Tree Data Structure

- **Binary tree**: In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees.
- **Ternary Tree**: A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as “left”, “mid” and “right”.
- **N-ary Tree or Generic Tree**: Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children(duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

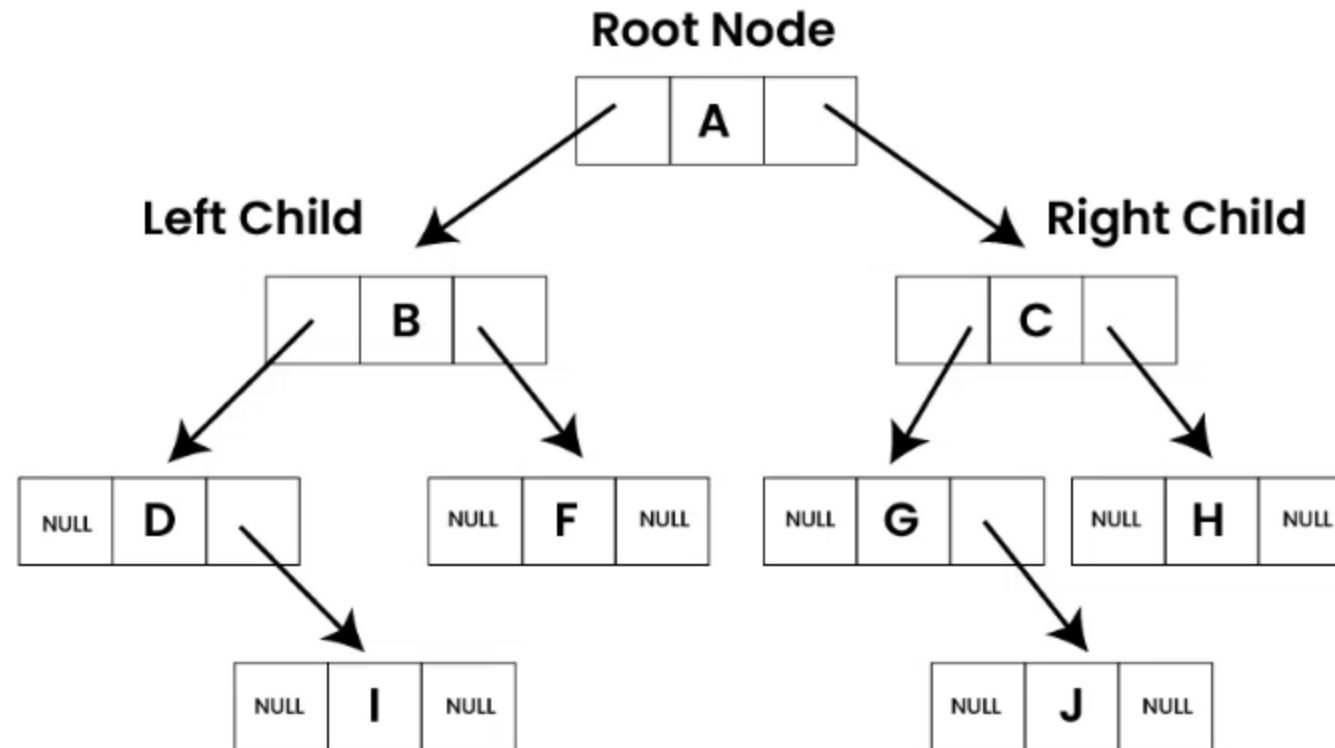
Applications of Tree Data Structure

- **File System:** This allows for efficient navigation and organization of files.
- **Data Compression:** Huffman coding is a popular technique for data compression that involves constructing a binary tree where the leaves represent characters and their frequency of occurrence. The resulting tree is used to encode the data in a way that minimizes the amount of storage required.
- **Compiler Design:** In compiler design, a syntax tree is used to represent the structure of a program.
- **Database Indexing:** B-trees and other tree structures are used in database indexing to efficiently search for and retrieve data.

Binary Tree

- Binary tree is a **tree data structure(non-linear)** in which each node can have **at most two children** which are referred to as the **left child** and the **right child**.
- The topmost node in a binary tree is called the **root**, and the bottom-most nodes are called **leaves**. A binary tree can be visualized as a hierarchical structure with the root at the top and the leaves at the bottom.

Representation of Binary Tree



- // Use any below method to implement Nodes of tree
- // Method 1: Using "struct" to make
- // user-define data type
- struct node {
- int data;
- struct node* left;
- struct node* right;
- };
- // Method 2: Using "class" to make
- // user-define data type
- class Node {
- public:
- int data;
- Node* left;
- Node* right;
- };

Insertion in Binary Tree

- Check if there is a node in the binary tree, which has missing left child. If such a node exists, then insert the new node as its left child.
- Check if there is a node in the binary tree, which has missing right child. If such a node exists, then insert the new node as its right child.
- If we don't find any node with missing left or right child, then find the node which has both the children missing and insert the node as its left or right child.

Deletion in Binary Tree

- Starting at the root, find the deepest and rightmost node in the binary tree and the node which we want to delete.
- Replace the deepest rightmost node's data with the node to be deleted.
- Then delete the deepest rightmost node.

Searching in Binary Tree

- Start from the root node.
- Check if the current node's value is equal to the target value.
- If the current node's value is equal to the target value, then this node is the required node.
- Otherwise, if the node's value is not equal to the target value, start the search in the left and right child.
- If we do not find any node whose value is equal to target, then the value is not present in the tree.

Binary Search Tree

- **Binary Search Tree (BST)** is a special type of binary tree in which the left child of a node has a value less than the node's value and the right child has a value greater than the node's value. This property is called the BST property and it makes it possible to efficiently search, insert, and delete elements in the tree.

Basic Operations

- **Search** – Searches an element in a tree.
- **Insert** – Inserts an element in a tree.
- **Pre-order Traversal** – Traverses a tree in a pre-order manner.
- **In-order Traversal** – Traverses a tree in an in-order manner.
- **Post-order Traversal** – Traverses a tree in a post-order manner.

creating a binary search tree

- 45, 15, 79, 90, 10, 55, 12, 20, 50